TECHNICAL ARCHITECTURE REPORT · LANGCHAIN FRAMEWORK

# RAG Pipeline Components

A comprehensive reference on the five foundational stages of a Retrieval-Augmented Generation system — from raw document ingestion to semantic retrieval — built with LangChain.

- Data Ingestion
- Text Splitting
- Embedding
- Vector Storage
- Retrieval

CONTENTS



## ARCHITECTURE OVERVIEW

# The Full RAG Pipeline

Retrieval-Augmented Generation (RAG) augments a large language model with real-time access to an external knowledge base. Instead of relying solely on parameters learned during training, the model is supplied with retrieved context that is directly relevant to the query.

LangChain provides composable abstractions for each of the five pipeline stages, allowing developers to swap implementations — for example, exchanging `RecursiveCharacterTextSplitter` for `SemanticChunker` — without rewriting the surrounding logic.

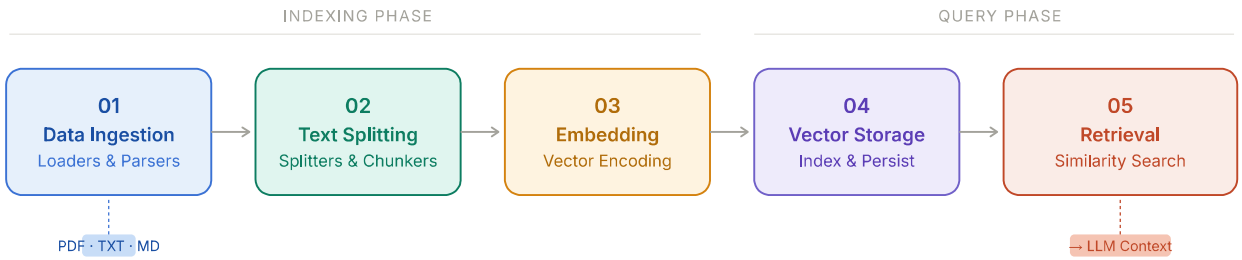


FIGURE 1 — FULL RAG PIPELINE: INDEXING PHASE (LEFT) + QUERY PHASE (RIGHT)

> **Key insight:** Stages 1–4 (ingestion → storage) form the *offline indexing phase* — run once (or periodically). Stage 5 (retrieval) is the *online query phase* — executed on every user request, where latency matters most.

---

STAGE 01 · DATA INGESTION

# Loading Documents into LangChain

The pipeline starts with raw, unstructured sources: PDFs, web pages, databases, APIs, Markdown files, and more. LangChain's **Document Loaders** normalize all of these into a uniform `Document` object that carries both `page_content` (the text) and `metadata` (source, author, timestamps).

## What is a Document?

Every piece of content flowing through LangChain is represented as a `Document`:

```python
from langchain.schema import Document

doc = Document(
    page_content="LangChain enables building LLM-powered applications.",
    metadata={
        "source": "docs/intro.pdf",
        "page": 1,
        "author": "Harrison Chase"
    }
)
```

## Core Document Loaders

LOADER · 01

### PyPDFLoader

Extracts text page-by-page from PDF files. Preserves page numbers in metadata for source tracing.

LOADER · 02

### WebBaseLoader

Fetches and parses HTML pages using BeautifulSoup. Handles article extraction and cleans boilerplate.

LOADER · 03

### TextLoader / CSVLoader

Reads plain text and tabular data. CSVLoader creates one Document per row with column-mapped metadata.

LOADER · 04

### DirectoryLoader

Recursively loads all files matching a glob pattern from a folder, using an inner loader for each type.

## Ingestion Flow Diagram

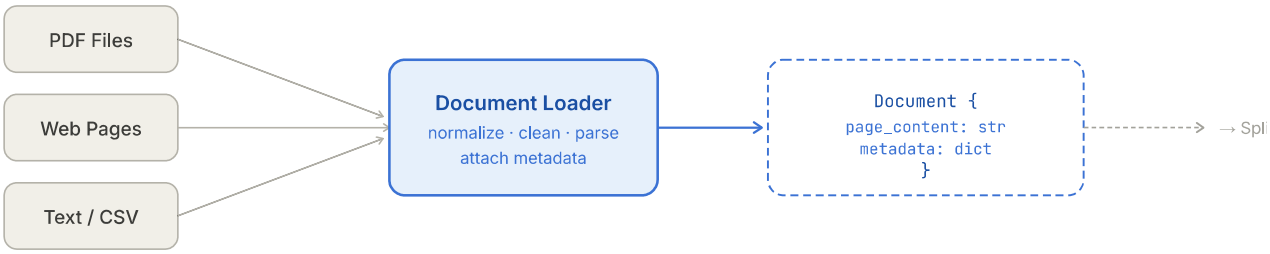


FIGURE 2 — DOCUMENT LOADER NORMALISATION FLOW

```python
# Example: load a PDF and a web page
from langchain_community.document_loaders import PyPDFLoader, WebBaseLoader

pdf_loader = PyPDFLoader("research_paper.pdf")
web_loader  = WebBaseLoader("https://example.com/article")

pdf_docs = pdf_loader.load()          # list[Document]
web_docs = web_loader.load()          # list[Document]

all_docs = pdf_docs + web_docs
```

---

STAGE 02 · TEXT SPLITTING

# Chunking for Retrieval Quality

LLMs have fixed context windows, and dense full-document retrieval is impractical. Text splitters break `Document` objects into smaller, semantically meaningful **chunks**. The granularity of these chunks directly determines retrieval precision — chunks that are too large waste context; chunks that are too small lose coherence.

### The Chunk Size / Overlap Trade-off

$$\text{effective coverage} = \text{chunk\_size} + \text{chunk\_overlap} \times (n - 1)$$

A non-zero `chunk_overlap` ensures that sentences or phrases that span a split boundary are represented in at least one complete chunk, preserving context continuity.

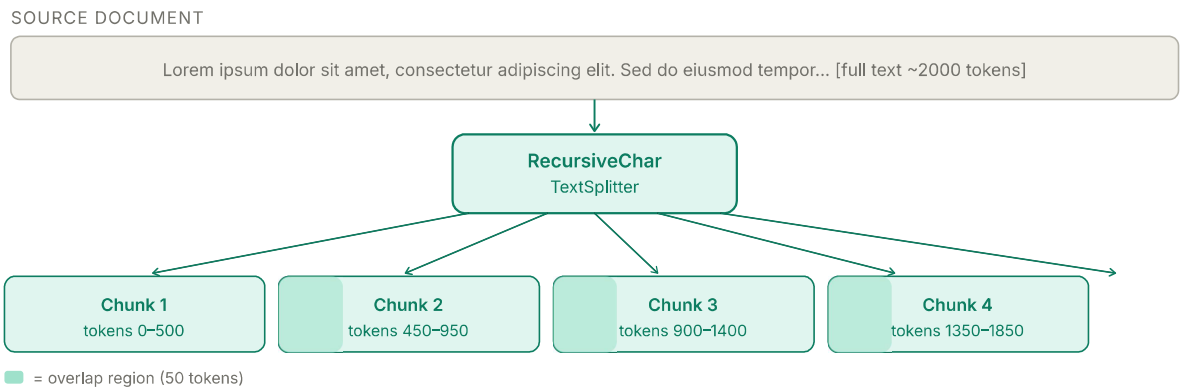


FIGURE 3 — RECURSIVECHARACTERTEXTSPLITTER PRODUCING OVERLAPPING CHUNKS

## Splitting Strategies

| Splitter | Strategy | Best For | Key Parameters |
|---|---|---|---|
| `RecursiveCharacterTextSplitter` | Splits on `["\n\n", "\n", " ", ""]` in order | General text — most documents | `chunk_size`, `chunk_overlap` |
| `CharacterTextSplitter` | Splits on a single separator character | Well-formatted text with known delimiters | `separator`, `chunk_size` |
| `TokenTextSplitter` | Splits by token count (tiktoken) | OpenAI model context budgeting | `encoding_name`, `chunk_size` |

| Splitter | Strategy | Best For | Key Parameters |
| --- | --- | --- | --- |
| `MarkdownHeaderTextSplitter` | Splits on # / ## / ### headers | Documentation, Notion exports, READMEs | `headers_to_split_on` |
| `SemanticChunker` | Embeds sentences, splits on cosine-distance breakpoints | High-quality QA where precision matters | `breakpoint_threshold_type` |

```
from langchain.text_splitter import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=500,
    chunk_overlap=50,
    separators=["\n\n", "\n", " ", ""]
)

chunks = splitter.split_documents(all_docs)
print(len(chunks))   # e.g. 84 chunks from a 40-page PDF
```

---

STAGE 03 · EMBEDDING

# Converting Text to Vectors

Embeddings transform text chunks into dense numeric vectors in a high-dimensional space (typically 768–3072 dimensions). Semantically similar chunks cluster together; dissimilar chunks lie far apart. This geometric relationship is what makes semantic search possible.

### Cosine Similarity

The standard similarity metric between two embedding vectors is cosine similarity:

$$\text{sim}(\mathbf{q}, \mathbf{c}) = \frac{\mathbf{q} \cdot \mathbf{c}}{|\mathbf{q}|\,|\mathbf{c}|} = \frac{\sum_i q_i c_i}{\sqrt{\sum_i q_i^2} \cdot \sqrt{\sum_i c_i^2}}$$

A cosine similarity of **1.0** means identical direction (semantically equivalent), **0.0** means orthogonal (unrelated), and **−1.0** means semantically opposite.

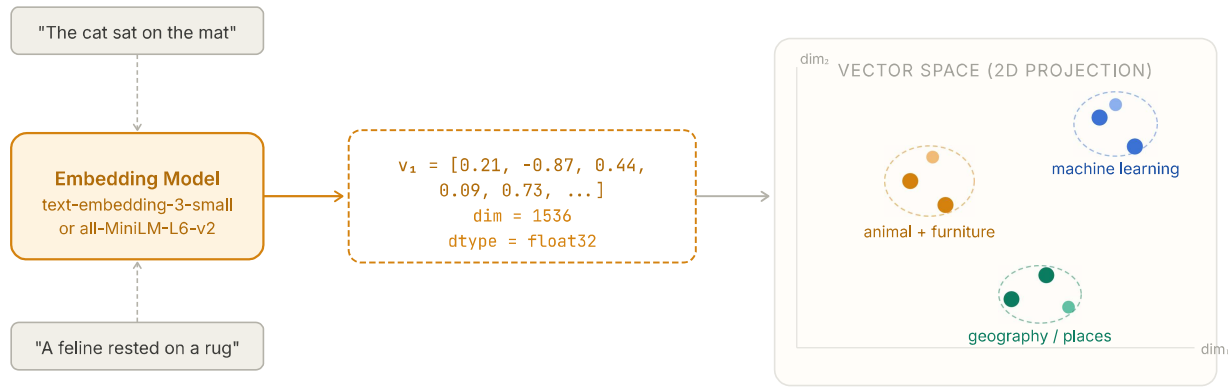


FIGURE 4 — TEXTS MAPPED TO VECTOR SPACE; SEMANTICALLY RELATED CHUNKS CLUSTER TOGETHER

## Common Embedding Models

PROVIDER · OpenAI

### text-embedding-3-small

1536-dim output, cost-efficient, strong multilingual support. Preferred for production RAG with OpenAI stack.

PROVIDER · HuggingFace

### all-MiniLM-L6-v2

384-dim, runs locally for free. Excellent for privacy-sensitive or on-premise deployments.

PROVIDER · Cohere

### embed-english-v3.0

1024-dim with input_type differentiation (search_document vs search_query) — improves retrieval precision.

```
from langchain_openai import OpenAIEmbeddings

embeddings = OpenAIEmbeddings(model="text-embedding-3-small")
```

```python
# Embed a single string
vector = embeddings.embed_query("What is LangChain?")
print(len(vector))  # 1536

# Embed all chunks (batch call)
vectors = embeddings.embed_documents([c.page_content for c in chunks])
```

---

STAGE 04 · VECTOR STORAGE

# Indexing Vectors for Fast Search

Once chunks are embedded, the vectors must be persisted in a data structure that supports efficient approximate nearest-neighbour (ANN) search at query time. LangChain's **VectorStore** abstraction wraps dozens of databases behind a uniform interface.

## In-memory vs Persistent vs Managed

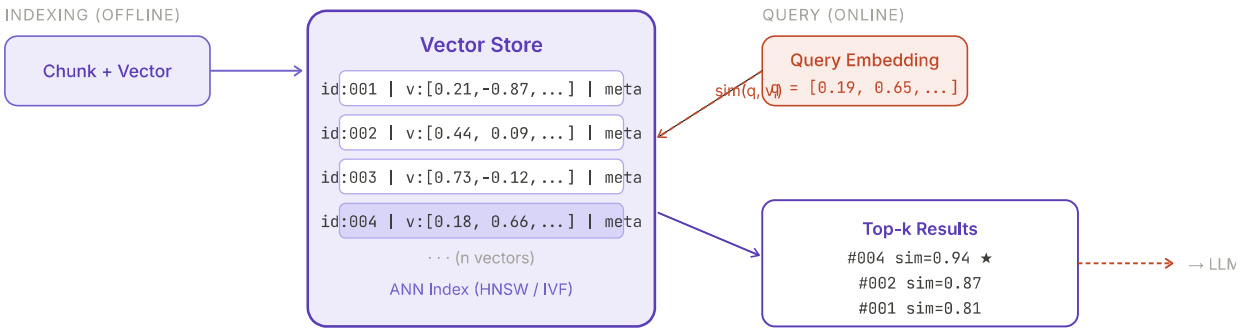


FIGURE 5 — VECTOR STORE: INDEXED EMBEDDINGS SEARCHED BY QUERY VECTOR AT RUNTIME

## VectorStore Options

| Store | Type | ANN Index | Best For |
|---|---|---|---|
| `FAISS` | In-memory / local disk | IVF + HNSW | Prototyping, offline batch |
| `Chroma` | Embedded (local SQLite) | HNSW | Dev + small production |
| `Pinecone` | Managed cloud | Proprietary ANN | Scalable SaaS production |
| `Weaviate` | Self-hosted / cloud | HNSW | Hybrid search + filtering |
| `pgvector` | PostgreSQL extension | IVF | SQL-native apps, joins |

```python
from langchain_community.vectorstores import FAISS
from langchain_openai import OpenAIEmbeddings

embeddings = OpenAIEmbeddings()

# Build index from chunks
vectorstore = FAISS.from_documents(chunks, embeddings)

# Persist to disk
vectorstore.save_local("faiss_index")

# Reload later
vectorstore = FAISS.load_local("faiss_index", embeddings)
```

---

STAGE 05 · RETRIEVAL

# Fetching the Right Context at Query Time

At inference time, the user's query is embedded using the *same model* used during indexing, and the resulting vector is compared against all stored chunk vectors. The top-*k* most similar chunks are returned as context and passed to the LLM's prompt.

### The Retrieval Query Loop

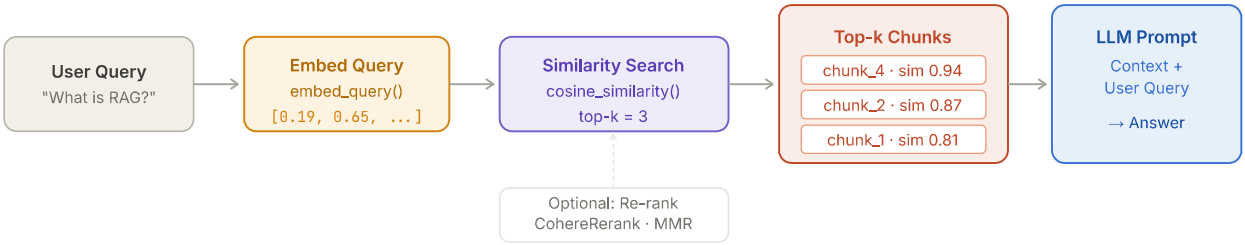


FIGURE 6 — FULL RETRIEVAL QUERY LOOP AT INFERENCE TIME

## Retrieval Methods

METHOD · 01

### Similarity Search

Returns the top-k chunks by cosine similarity score. Fast and effective for most use cases.

METHOD · 02

### MMR (Maximal Marginal Relevance)

Balances relevance with diversity — penalises returning similar chunks, improves answer coverage.

METHOD · 03

### Self-Query Retriever

LLM automatically generates both the semantic query and metadata filters from the natural language question.

METHOD · 04

### Multi-Query Retriever

Generates multiple paraphrases of the query, retrieves for each, and deduplicates results for broader recall.

```
# Basic retriever
retriever = vectorstore.as_retriever(
    search_type="similarity",
    search_kwargs={"k": 4}
)
```

```python
# MMR retriever (diverse results)
retriever_mmr = vectorstore.as_retriever(
    search_type="mmr",
    search_kwargs={"k": 5, "fetch_k": 20}
)

# Wire into a RetrievalQA chain
from langchain.chains import RetrievalQA
from langchain_openai import ChatOpenAI

qa_chain = RetrievalQA.from_chain_type(
    llm=ChatOpenAI(model="gpt-4o-mini"),
    retriever=retriever,
    return_source_documents=True
)

result = qa_chain.invoke({"query": "What is Retrieval-Augmented Generation?"})
print(result["result"])
```

**Production tip:** Set `search_kwargs={"k": 3, "score_threshold": 0.75}` to filter out low-confidence chunks. Returning irrelevant context is worse than returning no context — the LLM may hallucinate by synthesising conflicting chunks.

---

SUMMARY · COMPONENT COMPARISON

# Choosing the Right Tools

Below is a quick-reference guide for selecting implementations at each pipeline stage, organised by use-case maturity.

| Stage | Development / Prototype | Production | Key Metric |
|---|---|---|---|
| **Ingestion** | `TextLoader`, `PyPDFLoader` | `UnstructuredFileLoader`, cloud pipelines | Parse accuracy, throughput |

| Stage | Development / Prototype | Production | Key Metric |
|---|---|---|---|
| **Splitting** | `RecursiveCharacterTextSplitter` | `SemanticChunker`, `MarkdownHeaderTextSplitter` | Chunk coherence, recall@k |
| **Embedding** | `all-MiniLM-L6-v2` (free, local) | `text-embedding-3-small`, `embed-english-v3` | MTEB benchmark score |
| **Storage** | `FAISS` (in-memory), `Chroma` | `Pinecone`, `Weaviate`, `pgvector` | ANN latency p99, QPS |
| **Retrieval** | Similarity search, `k=4` | MMR + re-ranking (`CohereRerank`) | MRR, NDCG@k |

> **End-to-end tip:** Use LangSmith tracing (`LANGCHAIN_TRACING_V2=true`) to profile each stage in production. The most common bottlenecks are over-sized chunks (low precision) and insufficient overlap (missing context at split boundaries).

**Uditya Narayan Tiwari**

Deep Learning Practitioner & Technical Author

Portfolio GitHub LinkedIn Knowledge Base